ALERT-99: Best-Practice Summary and Setup Checklist

> **Series:** ALERT — Alerting Strategy and Design | **Notebook:** 99 | **Created:** June 2026 | **Last Updated:** 10/23/2026

Overview

The series on one page: the principles, a complete end-to-end setup checklist, and the cross-series map of where each piece is built. Use this as the standing reference once you have read ALERT-01 through ALERT-04.

Table of Contents

1. Principles
2. Complete Setup Checklist
3. The Ongoing Audit Loop
4. Cross-Series Map

Prerequisites

Requirement	Details
Prior reading	ALERT-01 through ALERT-04
Companion series	AIOPS, SLO, WFLOW, OPIPE

1. Principles

1. **OOTB Davis first.** Tune before you build; descend the funnel only when the layer above cannot express the condition.
2. **One enriched problem is the hub.** Every mechanism converges on a Davis problem; everything downstream operates on it.
3. **Enrich upstream or you cannot route downstream.** Team/area/severity belong on the problem before a workflow ever sees it.
4. **Burn-rate beats threshold-on-SLI.** Alert on how fast the budget drains, with multiwindow confirmation.
5. **Prefer simple workflows.** Pay for multi-step only when you need conditional/multi-team logic.
6. **Match destination to urgency.** Fast-burn → page; slow-burn → ticket.
7. **Validate every query before it becomes an alert.** A detector on a silent query is worse than no alert.
8. **Audit on a cadence.** Building well is half the job. A standing review — classify every problem, trace each false positive to the config behind it, and track the trend per area — keeps the funnel honest as services change. The 0.1%-of-observed-time yardstick in §3 is what turns “too noisy” from an opinion into a measurement.

2. Complete Setup Checklist

Detection

- [ ] Reviewed OOTB Davis anomaly detection settings and tuned sensitivity
- [ ] Custom anomaly detectors only where Davis does not cover the condition (auto-adaptive/seasonal, not static-on-traffic)
- [ ] Log/span signals extracted to OpenPipeline metrics before alerting on them
- [ ] SLOs defined for critical user journeys with burn-rate alerts
- [ ] New detectors deployed with relaxed thresholds first, tightened after 1–2 weeks of observation
- [ ] New detectors shadow-deployed as `CUSTOM_INFO` and measured before promotion to `CUSTOM_ALERT`
- [ ] Detectors alert on aggregated data, not split by high-cardinality dimensions (version, pod, status code)
- [ ] Sparse log/event conditions use records-based detectors with a large window and matching delay

Enrichment
- [ ] Detector event templates carry team / area / service properties
- [ ] Primary Grail tags (`primary_tags.*`) set for the routing dimensions — they carry onto the problem, as a deduplicated union across its entities
- [ ] Entity tags and Smartscape ownership populated for OOTB problems (a workflow reads ownership with the `get_owners` action, which is a second task and so a standard workflow)
- [ ] `event.severity` used to drive priority, and each event source confirmed to actually *set* it: from SaaS 1.348 (rollout start 10/12/2026, staged tenant rollout) Davis events and problems no longer default it to 3, so severity filters and priority mappings need a plan for a missing value (ALERT-04 §5)
- [ ] Custom detector event templates set `dt.smartscape_source.id` to an existing entity ID, as the docs recommend — custom alerts raised without one have been seen attached to the environment entity alone (ALERT-02 §3)

Routing
- [ ] Problem-trigger workflows filter on enriched metadata
- [ ] Simple workflows used wherever conditional logic is not needed
- [ ] Fast-burn → page channel; slow-burn → ticket channel
- [ ] One notification per problem (no duplicate alerts on raw metrics)

ITSM / destinations
- [ ] ServiceNow incidents carry assignment group, priority, and a de-dup correlation key
- [ ] Legacy alerting-profile integrations identified; migration to workflows considered

Governance
- [ ] Production detectors/SLOs promoted to config-as-code
- [ ] Alerts that fire without action reviewed and tuned
- [ ] Quarterly removal pass run: never-fired detectors, orphaned entities, and problems open for weeks

3. The Ongoing Audit Loop

The setup checklist above is build-time. Noise creeps back as services, traffic, and ownership change, so a standing audit is what keeps the funnel honest. The point is not to *count* false positives — it is to find the repeating configuration patterns behind them, so the same noise is not regenerated next period.

The yardstick: 0.1% of observed time. A healthy service should sit in an alert state no more than 0.1% of the time it is observed — about **1 to 1.5 minutes a day**, or roughly **43**

minutes a month. For a well-calibrated detector that means firing once or twice a *month*, not on a daily drumbeat.

The value of the number is not its precision. It is that it converts "this feels noisy" into something anyone can check: a detector firing several times a week is a tuning candidate by definition rather than by opinion, which is what lets you take the finding to an owning team without it becoming an argument. AIOPS-02 §8 carries the triage query that ranks detectors by firing count and shows, per detector, whether it is attributable and whether you can navigate to its configuration.

Classify every problem. Each problem in the audit window lands in exactly one bucket:

Class	Meaning	Action
True Positive (TP)	A real issue that required action	None — the detector is working
False Positive (FP)	Fired but needed no action — a threshold, scope, detection-method, or sliding-window issue	Trace it to the config and fix the source
Under Review (UR)	Not yet conclusive	Carry to the next period; gather more data points

Every FP must trace back to a concrete cause — the analyzer type, the entity scope, the sliding window, or the event template. An FP with no configuration change behind it is just a number; an FP with an identified root cause is progress.

Track one KPI: the false-positive rate over time, per area, and whether it is trending down. That single trend is the primary output of every cycle.

Signals to scan each cycle:

Signal	What it flags
Fire rate	A detector firing many times a day is a tuning or de-dup candidate
Silence rate	Never fired in 90+ days → redundant, or watching a dead path
Immediate-silence rate	Acknowledged and closed with no action = noise
Routing coverage	Every alert should route to exactly one owner; an unrouted alert is invisible
Runbook link	An alert with no runbook attached has no possible action

Signal	What it flags
Owner / area property	Present, so the alert maps to a team — also catches copy-paste clones that never set it
Problem age	A problem open for weeks is not a long incident — it is a misconfigured detector or an entity that no longer exists
Correlation coverage	Alerts attached to the environment entity alone have nothing more specific to correlate on and collect in large problems. Custom alerts without `dt.smartscape_source.id` are one cause; an integration that reports against the environment is another

Remove, do not only tune. A review that only ever adjusts thresholds accumulates configuration. Each quarter, actively delete:

- Detectors firing more than a few times a week that nobody has acted on — noise carrying a maintenance cost.
- Detectors that have not fired in months. Either the condition cannot occur or something else is catching it; either way the configuration is dead weight, and its silence is indistinguishable from coverage.
- Alerts bound to orphaned entities — decommissioned services, retired application versions, hosts long gone.
- Detectors behind problems that have stayed open for weeks. Disabling one closes its problem and stops the wasted re-evaluation.

Cadence:

Stage	When	Focus
Pre-go-live	Before an area or service reaches production	Every new or migrated detector fires correctly in non-prod; nothing fires on no-data
Early (first ~3 months)	Monthly	Top-noise alerts reviewed with the owning team
Steady state	Quarterly	Cross-area review; retire detectors silent 90+ days or at a 100% silence rate

Express these as recurring queries, and pick the data object deliberately — there are **three** surfaces here and they answer different questions. Counts measured over the same 7 days on one tenant, 10/06/2026:

Query	Returns	Use it for
`fetch dt.davis.problems`	4,246	**Problem-level signals** — one row per problem. MTTR, severity rollups, problem age.
`fetch dt.davis.events`	264,272	**Detector-level noise ranking** — the firing stream (AIOPS-02 §8).
`fetch events | filter event.kind == "DAVIS_PROBLEM"`	122,837	The **raw problem-event stream** — many records per problem as its state updates. Valid, but do not count these and call the result "problems": they cover 4,264 distinct problems, a 29× over-count. The multiplier is not a constant to divide by — on the same tenant it was about 750× on 07/31/2026.

The trap worth naming: **`fetch dt.davis.events | filter event.kind == "DAVIS_PROBLEM"` returns exactly zero** — `dt.davis.events` carries only `DAVIS_EVENT` (AIOPS-03 §3). That is a filter on the wrong table, not a deprecated form, and it fails silently. The superficially similar `fetch events | filter event.kind == "DAVIS_PROBLEM"` is a different query against a different table and works fine. AIOPS-03 and ORGNZ-10 carry validated problem-feed patterns to build on.

Two of the signals above are worth turning into standing trend queries rather than one-off checks, both built from the two-data-object model in AIOPS-03 §3.

Denoising ratio (events → problems). How much is Causal AI's grouping actually consolidating? Divide problem-worthy events by the problems they produced, trended daily — a ratio sitting near 1 for a stretch means events are arriving as separate problems rather than merging. Read it alongside "Correlation coverage" above rather than as the same signal: poor correlation coverage pushes the ratio the *other* way. On the tenant above, problems attached only to the environment grouped 8.5 events each, against about one for every other problem (7 days to 10/06/2026). Filter both sides of the ratio on the same event categories, or problems from an excluded category enter the denominator with no events behind them. As with the 0.1% yardstick, there's no universal healthy number here; trend it against your own baseline.

```
// Denoising ratio — problem-worthy events divided by resulting problems,
trended daily
//
// Both streams must be confined to the SAME window. dt.davis.problems is
fetched by
// record timestamp, but binned by event.start — so a problem that started
before the
// window (or is still open from weeks ago) lands in a day bucket with zero
events and
// reports denoising_ratio = 0. Verified 08/27/2026: without the event.start
filter the
// first 12 rows were all such artifacts, dated back to 2026-06-01.
//
// Both streams must also use the SAME categories. Filtering only the events
would leave
// problems opened by excluded categories (custom alerts, for instance) in the
denominator
// with no events behind them, and the ratio would read low. Validated
10/06/2026:
// 31 daily rows, ratio 1.69-3.65, no notifications.
fetch dt.davis.events, from:-30d
| filter in(event.category,
{"AVAILABILITY","ERROR","RESOURCE_CONTENTION","SLOWDOWN","CUSTOM_ALERT"})
| fieldsAdd kind = "event", day = bin(timestamp, 24h)
| append [
    fetch dt.davis.problems, from:-30d
    | filter not(dt.davis.is_duplicate)
    | filter event.start &gt;= now() - 30d
    | filter in(event.category,
{"AVAILABILITY","ERROR","RESOURCE_CONTENTION","SLOWDOWN","CUSTOM_ALERT"})
    | fieldsAdd kind = "problem", day = bin(event.start, 24h)
  ]
| summarize {event_count = countIf(kind == "event"), problem_count =
countIf(kind == "problem")}, by:{day}
| filter problem_count &gt; 0 and event_count &gt; 0
| fieldsAdd denoising_ratio = round(toDouble(event_count) / problem_count,
decimals: 2)
| sort day asc
```

Correlation coverage. The share of non-informational Davis events attached to something more specific than the environment entity. The docs ask for `dt.smartscape_source.id` to be an existing entity ID so that the correlation engine can resolve the entity; they do not describe what happens to an event without one. The data shows what to watch: an event attached only to the environment has nothing but the environment to be grouped on, and those events collect in large problems (ALERT-02 §3). Measure the attachment itself, not whether `dt.smartscape_source.id` is present. On the

same tenant over 7 days, 88% of events without that field were still attached to a real entity, and one cloud integration's events carried a source yet were attached to the environment alone. Over 30 days to 10/06/2026 the daily share below ranged from 74% to 87%:

```
// Correlation coverage — share of non-informational Davis events attached to
something
// other than the environment entity alone, trended daily.
// affected_entity_ids is populated on every Davis event;
smartscape.affected_entities was
// present on fewer than half of them. Validated 10/06/2026: 31 daily values,
73.7-87.3%,
// no notifications.
fetch dt.davis.events, from:-30d
| filter not(in(event.category, {"INFO","WARNING"}))
| fieldsAdd env_only = arraySize(affected_entity_ids) == 1 and
startsWith(toString(affected_entity_ids[0]), "ENVIRONMENT-")
| fieldsAdd attributed_flag = if(env_only, 0.0, else: 100.0)
| makeTimeseries correlation_coverage_pct = avg(attributed_flag), interval:24h
```

When the share drops, break the environment-only events down by what produced them:

```
// What produces the environment-only events? A CUSTOM_* type with no source
points at
// that detector's event template; a built-in provider points at the
integration instead.
// Validated 10/06/2026: 6 rows, no notifications.
fetch dt.davis.events, from:-30d
| filter not(in(event.category, {"INFO","WARNING"}))
| filter arraySize(affected_entity_ids) == 1 and
startsWith(toString(affected_entity_ids[0]), "ENVIRONMENT-")
| fieldsAdd has_source = isNotNull(dt.smartscape_source.id)
| summarize {events = count(), detectors =
countDistinctExact(dt.settings.object_id)}, by:{event.provider, event.type,
dt.settings.schema_id, has_source}
| sort events desc
| limit 20
```

Reading the two together. When correlation coverage falls, the breakdown above says where to look. A `CUSTOM_*` event type with no source points at that detector's event template (the Enrichment block of Section 2's checklist). A built-in provider whose events land on the environment points at that integration, not at a template you own. A falling denoising ratio *despite* stable correlation coverage points the other way, at scoping or tuning rather than attribution.

> **Sources:** Avoid overalerting (DT docs) — *"make sure to set the dt.smartscape_source.id field to an existing Smartscape entity ID, like a host or service entity ID rather than an arbitrary string"*; *"The same Smartscape entity rule groups all events that share the same dt.smartscape_source.id field value."* Coverage, breakdown, denoising and over-count figures are tenant measurements run 10/06/2026. **Derived:** the audit signals table extends the documented review targets with the correlation-coverage and problem-age checks implied by the entity-ID and long-running-alert guidance; the denoising-ratio and correlation-coverage trend queries apply the two-data-object model (AIOPS-03 §3) to already-documented fields as standing metrics.

4. Cross-Series Map

Piece	Built in
Anomaly detector mechanics (analyzer, tuning, event template)	AIOPS-02
Metric events / detector schemas as code	AIOPS-02 §6, AUTOM-05/06
OpenPipeline metric extraction	OPIPE, FAQ-09
SLIs, error budgets, burn-rate alerting	SLO-02 / 03 / 04
SLOs as code	SLO-05
Routing, conditions, escalation	WFLOW-03 / 04
HTTP / webhook actions	WFLOW-08
ServiceNow incident integration	ALERT-04
Cost / DPS framing	FINOPS, FAQ-09

> **Sources:** Alerting and notifications (DT docs), Workflows (DT docs). **Derived:** the checklist and funnel synthesise the ALERT, AIOPS, SLO, and WFLOW series into one operational sequence.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.